

DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

Submitted by	
Name	GANGAVARAM NIHARIKA
Roll no	AV.SC.U4CSE25155
year/sem/section	2nd year/BTech CSE-B / Semester-3
Credits	

Submitted to	Dr. B Raj Kumar
Designation	Assistant Professor (Sl.Gr)
Department	Department of Computer Science

INDEX

SI.NO	Lab questions	Page.no	Date
1.	WEEK 1: A. A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by "launch". B. The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years. C. An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. D. In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number. E. Write a recursive python program to print the first n terms of the Fibonacci series.	5-11	14-07-26
2.	WEEK 2: A. Write a program to perform linear search on a given input. B. AIM: Write a Python program to perform binary search on the given input. Verify by taking a sorted array and an unsorted array as a list.	12-15	21-07-26
3.	WEEK 3: A.Write a python program to implement bubble sort, where the input elements are taken from the user. B. Write a python program to implement insertion sort, where the input elements are taken from the user. C. Write a python program to implement selection sort, where the input elements are taken from the user.	16-20	01-08-26
4.	WEEK 4: A.Write a python program to implement merge sort, where the input elements are taken from the user.		

	B. Write a python program to implement quick sort, where the input elements are taken from the user.	21-23	11-08-26
5.	WEEK 5: Linked Lists Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	24-46	18-08-26
6.	WEEK 6: A.Write a Python program to implement stack operations such as push,pop,display using an array. B.Write a Python program to implement stack operations such as push,pop,display using a linked list.	47-51	25-08-26
7.	WEEK 7: A. Write a Python program to implement queue and its operations such as enqueue, dequeue, heap and display with a menu driven approach, implement using array and linked list data structures. B. Write a Python program to implement queue and its operations such as enqueue, dequeue, heap and display with a menu driven approach, implement using array and linked list data structures.	52-58	22-09-26
8.	WEEK 8: Binary Trees A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	59-66	29-09-26
9.	WEEK 9: Binary Search Trees Python program to implement Binary Search Tree (BST) operations.	67-71	06-10-26

10.	WEEK 10: AVL Tree Write a Python program to implement an AVL Tree with all its operations.	72-78	13-10-26
11.	WEEK 11: Hashing Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	79-82	27-10-26

WEEK 1

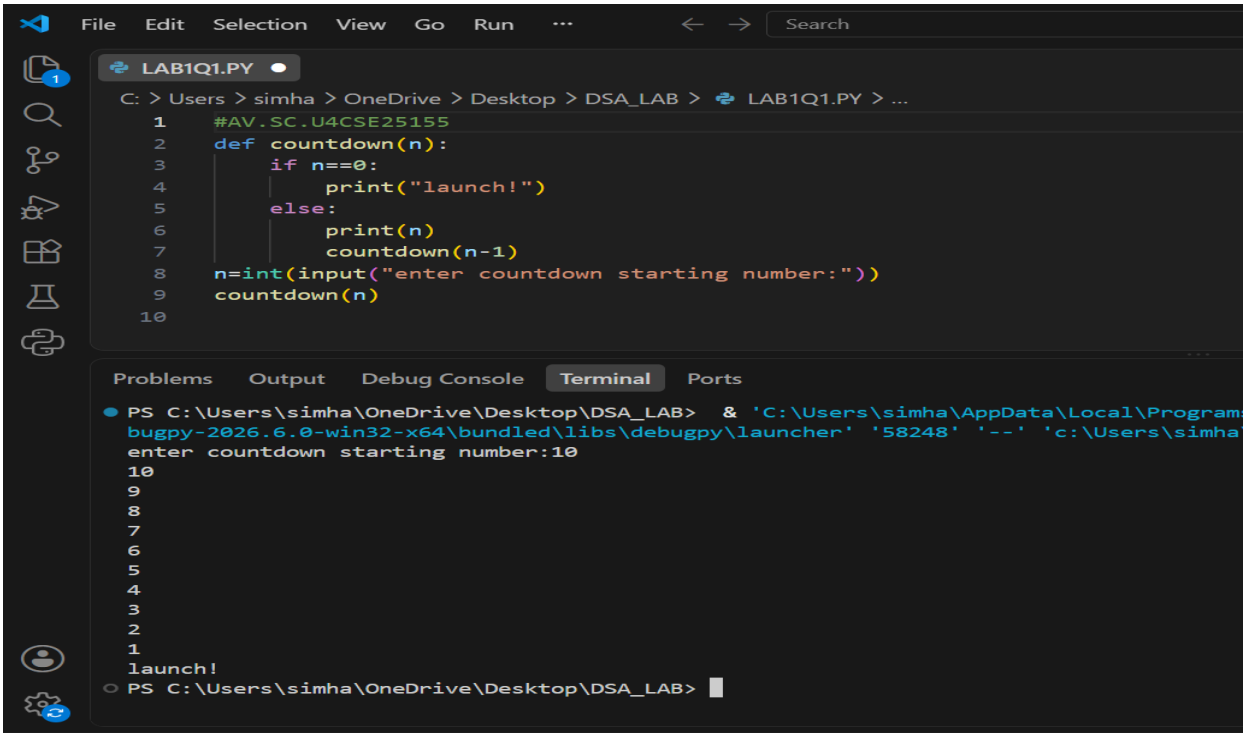
A.AIM:

A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def countdown(n):
    if n==0:
        print("launch!")
    else:
        print(n)
        countdown(n-1)
n=int(input("enter countdown starting number:"))
countdown(n)
```

OUTPUT:



EXPLAINATION:

This Python program uses **recursion** to create a countdown.

- `countdown(n)` is a recursive function.
- If `n == 0`, it prints **"launch!"** and stop.
- Otherwise, it prints `n` and calls itself with `n-1`.
- The `input()` function gets the starting number from the user.

ERROR TABLE:

S.NO	ERRORS	RECTIFICATIONS
1.	N==0	n==0

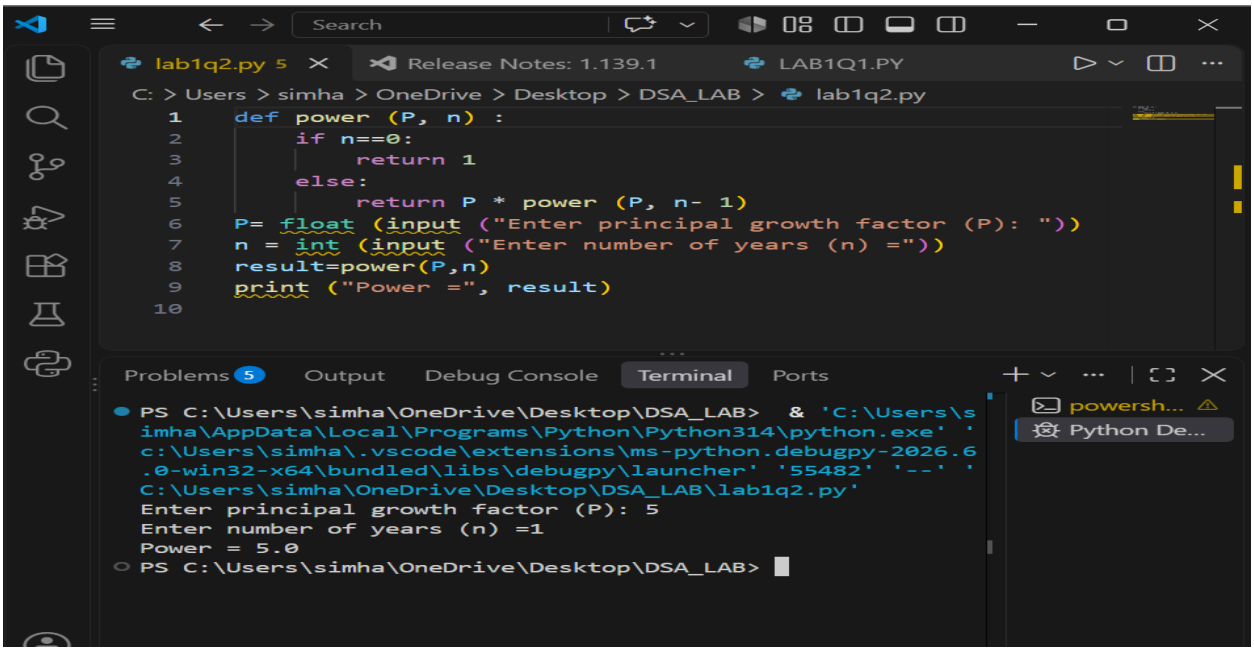
B. AIM:

The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
def power (P, n) :
  if n==0:
    return 1
  else:
    return P * power (P, n- 1)
P= float (input ("Enter principal growth factor (P): "))
n = int (input ("Enter number of years (n) ="))
result=power(P,n)
print ("Power =", result)
```

OUTPUT:



EXPLAINATION:

This Python program calculates P raised to the power n using recursion.

- power(P, n) is a recursive function.
- If n == 0, it returns 1 because any number raised to 0 is 1.
- Otherwise, it calculates P × power(P, n-1).
- The user enters P as a decimal number and n as an integer.
- Finally, the result is displayed as Power.

ERRORTABLE:

S.NO	ERRORS	RECTIFICATIONS
1.	indendation	Adding spaces

C. AIM:

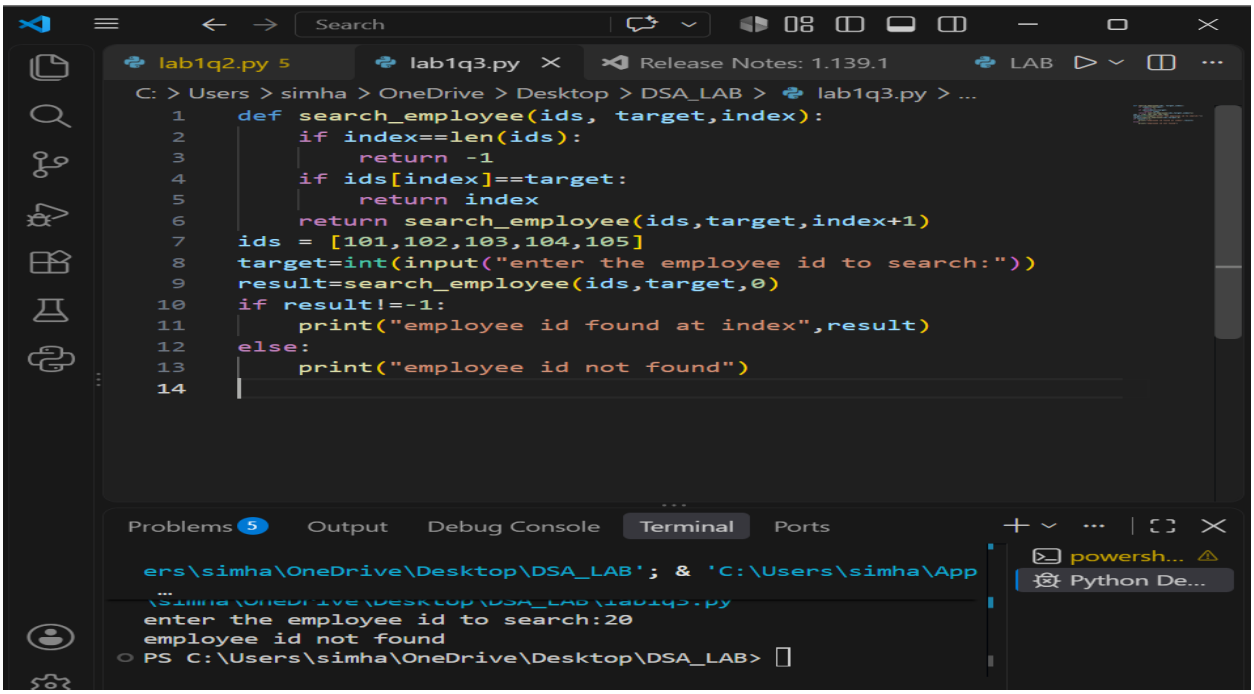
An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search_employee(ids, target,index):
    if index==len(ids):
```

```
        return -1
    if ids[index]==target:
        return index
    return search_employee(ids,target,index+1)
ids = [101,102,103,104,105]
target=int(input("enter the employee id to search:"))
result=search_employee(ids,target,0)
if result!=-1:
    print("employee id found at index",result)
else:
    print("employee id not found")
```

OUTPUT:



EXPLAINATION:

This Python program performs a recursive linear search for an employee ID.

- search_employee() checks each ID one by one.
- If the target ID is found, it returns its index.

- If the end of the list is reached, it returns -1.
- The program prints whether the employee ID was found or not

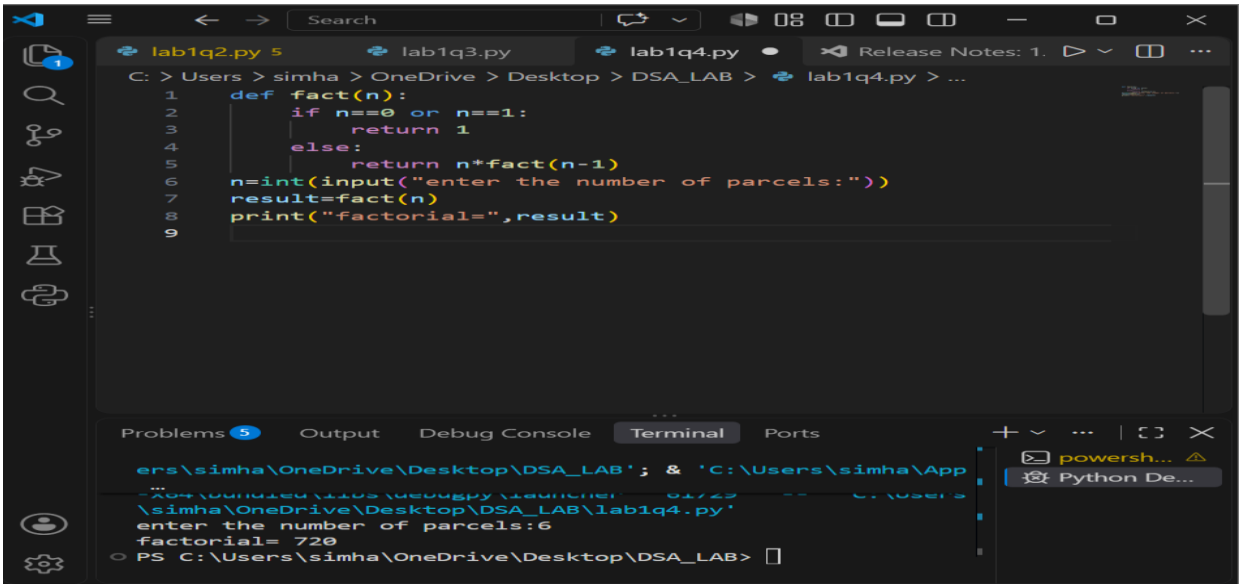
D. AIM:

In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def fact(n):
    if n==0 or n==1:
        return 1
    else:
        return n*fact(n-1)
n=int(input("enter the number of parcels:"))
result=fact(n)
print("factorial=",result)
```

OUTPUT:



EXPLAINATION:

This Python program calculates the **factorial of a number using recursion**.

- `fact(n)` is a recursive function.
- If `n` is `0` or `1`, it returns `1`.
- Otherwise, it calculates `n × fact(n-1)`.
- The user enters the number of parcels, and the factorial is displayed

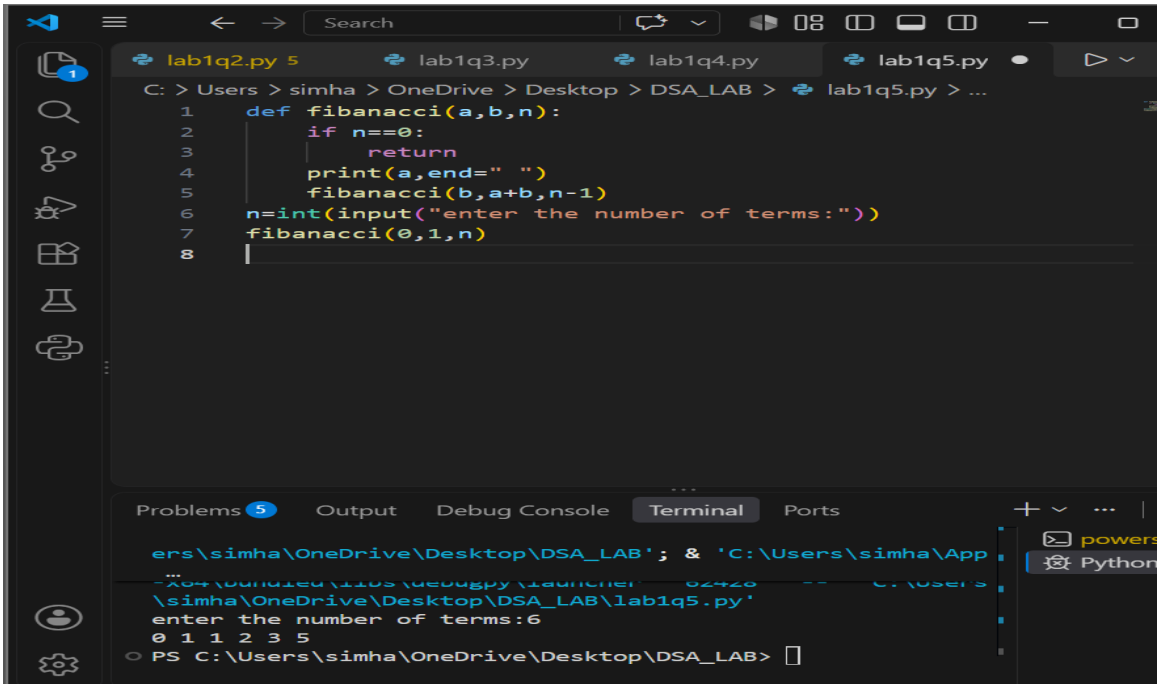
E. AIM:

Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibanacci(a,b,n):
    if n==0:
        return
    print(a,end=" ")
    fibanacci(b,a+b,n-1)
n=int(input("enter the number of terms:"))
fibanacci(0,1,n)
```

OUTPUT:



EXPLAINATION:

This Python program generates the **Fibonacci series using recursion**.

- `fibanacci(a, b, n)` prints the current Fibonacci number.
- It starts with `0` and `1`.
- The function calls itself with `b` and `a+b`.
- It stops when `n == 0`.
- The user enters the number of terms to print.

WEEK 2

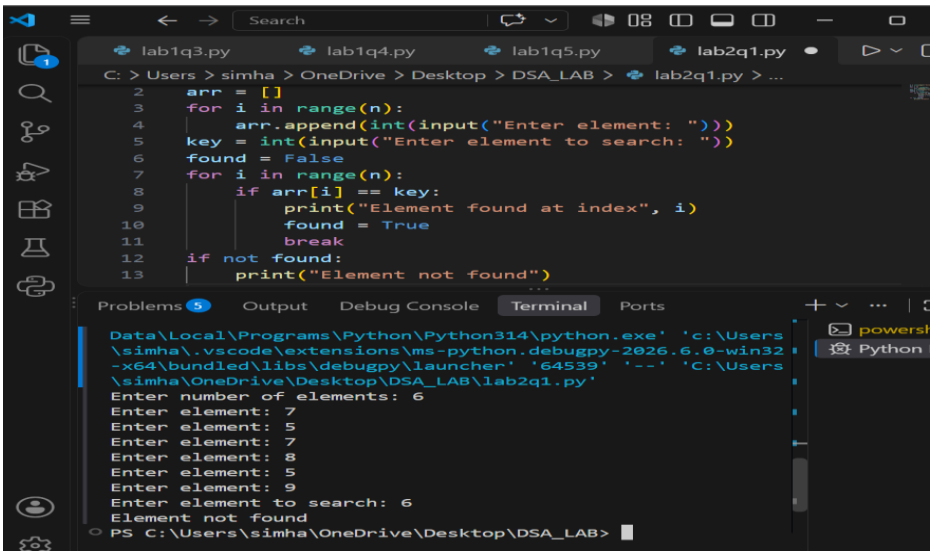
A. AIM:

Write a program to perform linear search on a given input.

PROGRAM:

```
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
key = int(input("Enter element to search: "))
found = False
for i in range(n):
    if arr[i] == key:
        print("Element found at index", i)
        found = True
        break
if not found:
    print("Element not found")
```

OUTPUT:



EXPLAINATION:

This Python program performs a **linear search** to find an element in an array.

- It takes the number of elements and stores them in `arr`.
- The user enters the element (`key`) to search for.
- The program checks each element one by one.
- If the element is found, it prints its **index** and stops searching.
- If it is not found, it prints **"Element not found"**

ERRORTABLE:

S.NO	ERRORS	RECTIFICATIONS
1.	indendation	Adding spaces

B. AIM:

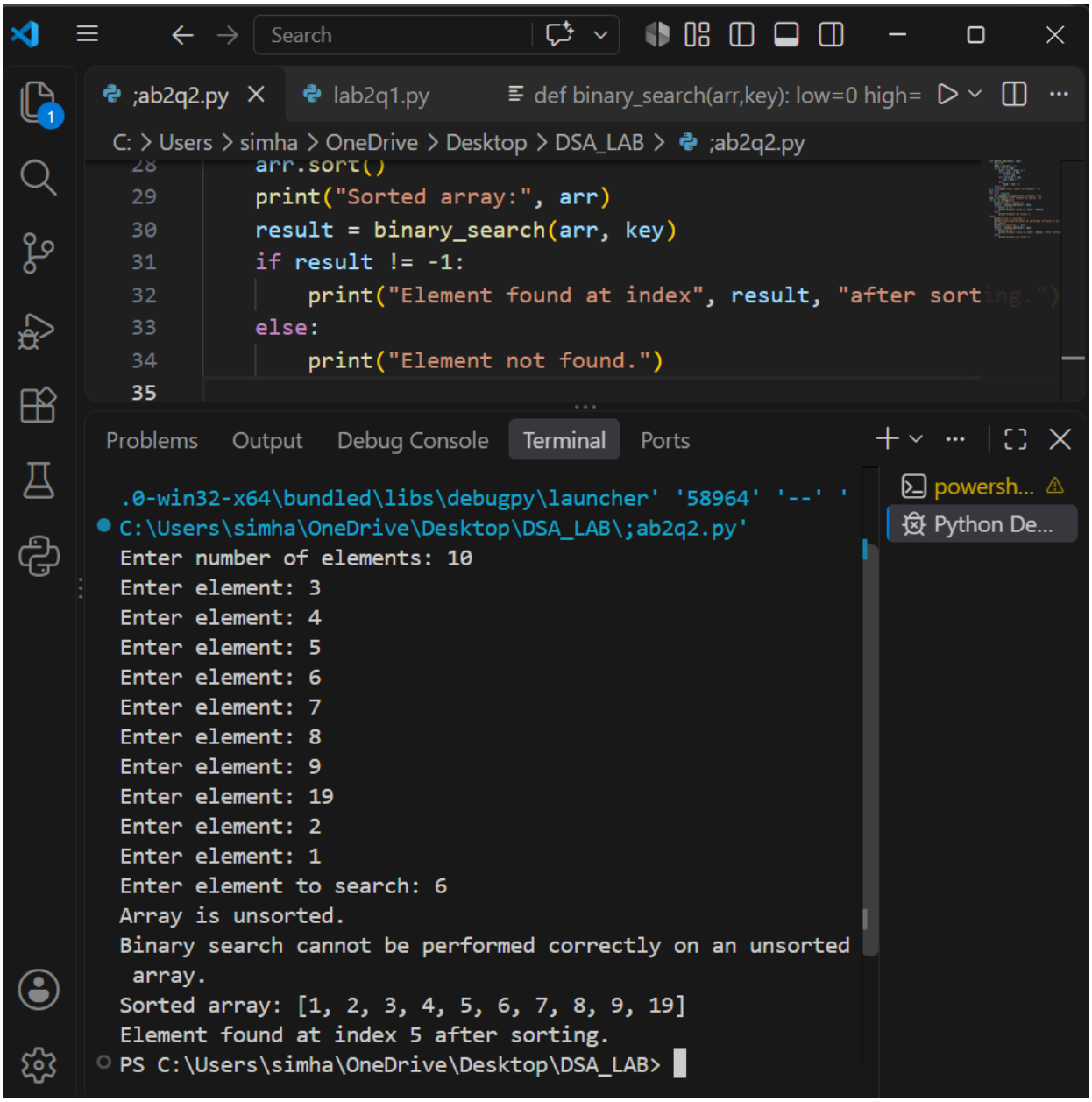
Write a Python program to perform binary search on the given input. Verify by taking a sorted array and an unsorted array as a list.

PROGRAM:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter number of elements: "))
```

```
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
key = int(input("Enter element to search: "))
if arr == sorted(arr):
    print("Array is sorted.")
    result = binary_search(arr, key)
    if result != -1:
        print("Element found at index", result)
    else:
        print("Element not found.")
else:
    print("Array is unsorted.")
    print("Binary search cannot be performed correctly on an unsorted array.")
    arr.sort()
    print("Sorted array:", arr)
    result = binary_search(arr, key)
    if result != -1:
        print("Element found at index", result, "after sorting.")
    else:
        print("Element not found.")
```

OUTPUT:



EXPLAINATION:

This Python program performs **binary search** on an array.

- It first takes the elements and the search key from the user.
- It checks whether the array is already sorted.
- If sorted, it directly performs **binary search**.
- If unsorted, it sorts the array first and then performs binary search.
- Binary search repeatedly divides the search range into two halves.
- It returns the index if the element is found; otherwise, it returns **–1.**

WEEK 3

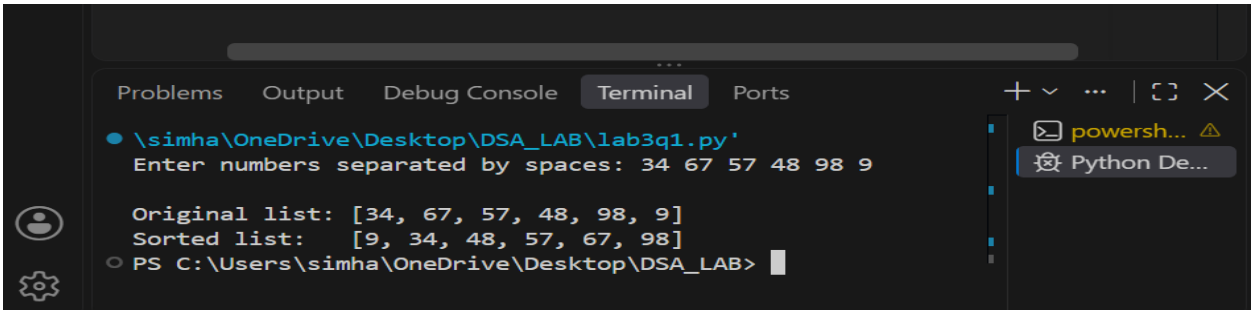
A.AIM:

Write a python program to implement bubble sort, where the input elements are taken from the user.

PROGRAM:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
user_input = input("Enter numbers separated by spaces: ")
numbers_list = [int(x) for x in user_input.split()]
print(f"\nOriginal list: {numbers_list}")
bubble_sort(numbers_list)
print(f"Sorted list:   {numbers_list}")
```

OUTPUT:



EXPLAINATION:

This Python program sorts a list of numbers using the **Bubble Sort algorithm**.

- It takes numbers from the user separated by spaces.
- `bubble_sort()` compares adjacent elements.
- If they are in the wrong order, they are swapped.
- This process continues until the list is sorted.
- `swapped` helps stop the loop early if the list is already sorted.

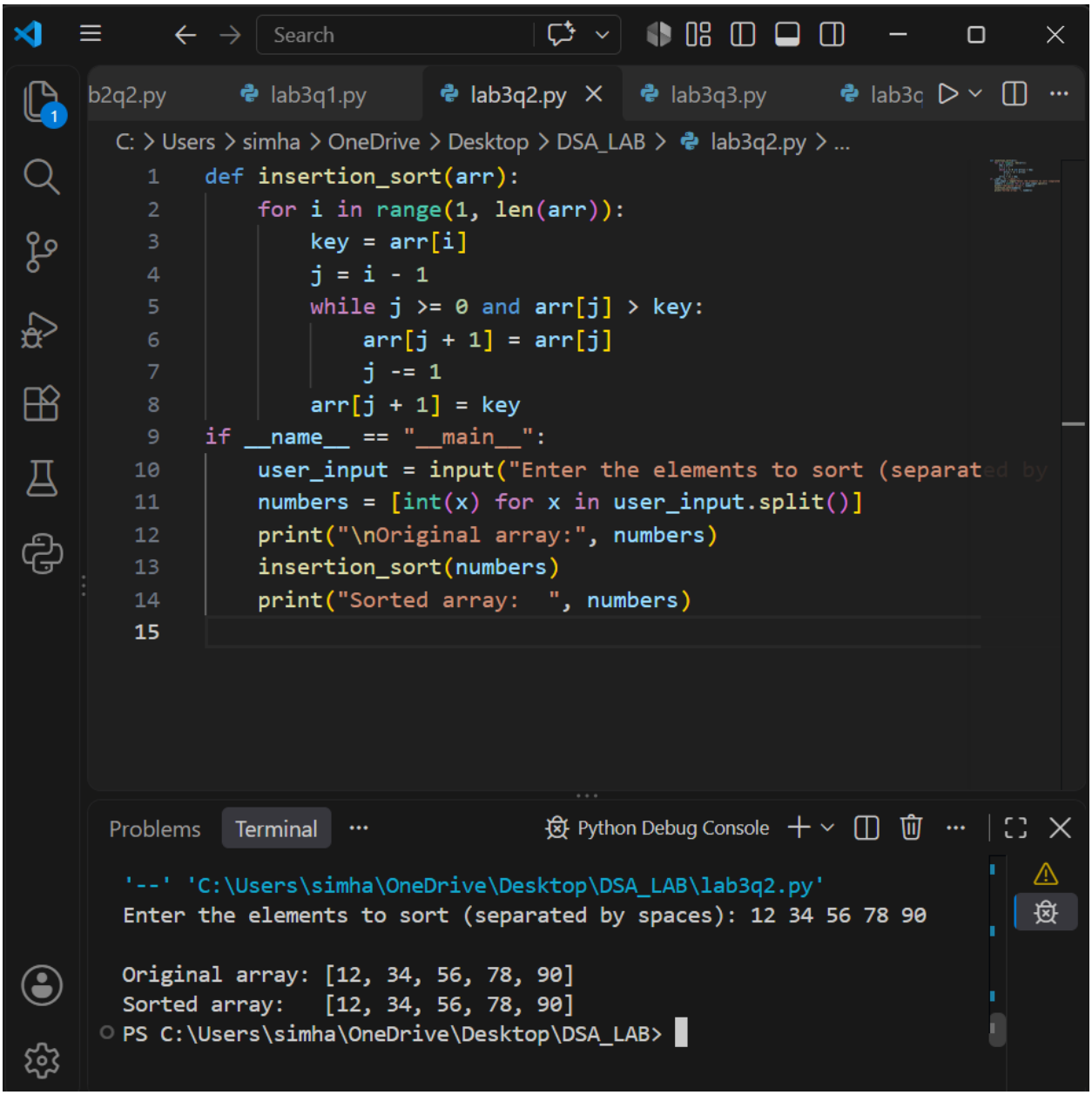
B.AIM:

Write a python program to implement insertion sort, where the input elements are taken from the user.

PROGRAM:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
if __name__ == "__main__":
    user_input = input("Enter the elements to sort (separated by spaces): ")
    numbers = [int(x) for x in user_input.split()]
    print("\nOriginal array:", numbers)
    insertion_sort(numbers)
    print("Sorted array:  ", numbers)
```

OUTPUT:



EXPLAINATION:

This Python program sorts a list of numbers using the **Insertion Sort algorithm**.

- It takes numbers from the user separated by spaces.
- Each element is compared with the elements before it.
- Larger elements are shifted one position to the right.
- The current element is inserted into its correct position.
- Finally, the sorted array is displayed.

ERRORTABLE:

s.no	Error	rectification
1.	ValueError	Enter only valid numbers
2.	TypeError	Make sure all elements are numbers

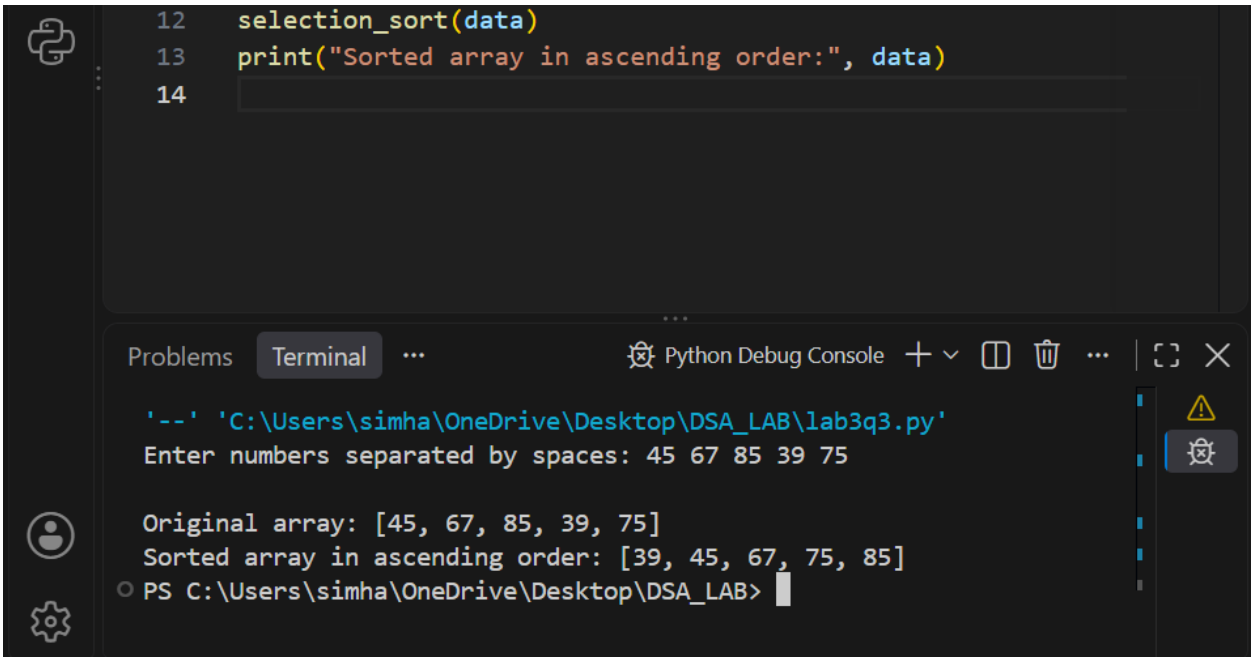
C.AIM:

Write a python program to implement selection sort, where the input elements are taken from the user.

PROGRAM:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
user_input = input("Enter numbers separated by spaces: ")
data = [int(x) for x in user_input.split()]
print("\nOriginal array:", data)
selection_sort(data)
print("Sorted array in ascending order:", data)
```

OUTPUT:



EXPLAINATION:

This Python program sorts a list of numbers using the **Selection Sort algorithm**.

- It takes numbers from the user separated by spaces.
- It finds the **smallest element** in the unsorted part of the list.
- It swaps the smallest element with the first unsorted element.
- This process continues until the entire list is sorted in **ascending order**.

WEEK 4

A.AIM:

Write a python program to implement merge sort, where the input elements are taken from the user.

PROGRAM:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
                k += 1
                while i < len(left):
                    arr[k] = left[i]
                    i += 1
                    k += 1
                while j < len(right):
                    arr[k] = right[j]
```

```
            j += 1
            k += 1
# Input from user
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
merge_sort(arr)
print("Sorted array:", arr)
```

OUTPUT:

```
= RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab4q1
.py
Enter elements separated by spaces: 56 67 87 98 45 34
Sorted array: [34, 56, 67, 87, 98, 45]
```

EXPLAINATION:

This Python program sorts an array using the **Merge Sort algorithm**.

- The array is divided into two halves recursively.
- Each half is sorted separately.
- The sorted halves are then **merged** to form one sorted array.
- Finally, the sorted array is displayed.

B.AIM:

Write a python program to implement quick sort, where the input elements are taken from the user.

PROGRAM:

```
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[high]
        i = low - 1
        for j in range(low, high):
            if arr[j] < pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
```

```
            arr[i + 1], arr[high] = arr[high], arr[i + 1]
            p = i + 1
            quick_sort(arr, low, p - 1)
            quick_sort(arr, p + 1, high)
# Input from user
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)
```

OUTPUT:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Pytho
n/Python314/lab4q2.py ==
Enter elements separated by spaces: 56 76 78 34 768
Sorted array: [56, 76, 34, 768, 78]
```

EXPLAINATION:

This Python program sorts an array using the **Quick Sort algorithm**.

- The last element is selected as the **pivot**.
- Elements smaller than the pivot are moved to the left.
- The pivot is placed in its correct position.
- The process is repeated recursively for the left and right parts.
- Finally, the sorted array is displayed.

ERRORTABLE:

S.NO	ERRORS	RECTIFICATIONS
1.	indendation	Adding spaces

WEEK 5

A.AIM:

Write a program to implement SLL with the following operations.

1. create linked list
2. insert beginning
3. insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/traverse
10. Exit

PROGRAM: class Node: def **init**(self, data): self.data = data self.next = None

class SinglyLinkedList: def **init**(self): self.head = None

```
# 1. Create linked list
def create(self):
    n = int(input("Enter number of nodes: "))

    for i in range(n):
        data = int(input("Enter data: "))
       new_node = Node(data)

       if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

    print("Linked list created successfully.")

# 2. Insert at beginning
```

```
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    new_node.next = self.head
    self.head = new_node

    print("Node inserted at beginning.")

# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    print("Node inserted at end.")

# 4. Insert at index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))

    if index < 0:
        print("Invalid index.")
        return

    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print("Node inserted.")
        return
```

```
        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range.")
            return

        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted.")

    # 5. Delete by value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("List is empty.")
            return

        if self.head.data == value:
            self.head = self.head.next
            print("Node deleted.")
            return

        temp = self.head

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not found.")
```

```
    else:
        temp.next = temp.next.next
        print("Node deleted.")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
    else:
        self.head = self.head.next
        print("First node deleted.")

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head.next is None:
        self.head = None
        print("Last node deleted.")
        return

    temp = self.head

    while temp.next.next:
        temp = temp.next

    temp.next = None
    print("Last node deleted.")

# 8. Count number of nodes
def count_nodes(self):
    count = 0
    temp = self.head

    while temp:
        count += 1
        temp = temp.next
```

```
    print("Number of nodes:", count)

# 9. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty.")
        return

    temp = self.head

    print("Linked List:", end=" ")

    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next

    print("None")
```

sll = SinglyLinkedList()

while True: print("\n----- SINGLY LINKED LIST -----") print("1. Create linked list") print("2. Insert at beginning") print("3. Insert at end") print("4. Insert at index") print("5. Delete by value") print("6. Delete first node") print("7. Delete last node") print("8. Count number of nodes") print("9. Display / Traverse") print("10. Exit")

```
choice = int(input("Enter your choice: "))
if choice == 1:
    sll.create()

elif choice == 2:
    sll.insert_beginning()

elif choice == 3:
    sll.insert_end()

elif choice == 4:
    sll.insert_at_index()
```

```
elif choice == 5:
    sll.delete_by_value()

elif choice == 6:
    sll.delete_first()

elif choice == 7:
    sll.delete_last()

elif choice == 8:
    sll.count_nodes()

elif choice == 9:
    sll.display()

elif choice == 10:
    print("Program terminated.")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab5
q1.py ==

----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 54
Node inserted at beginning.

----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 65
Enter data: 67
Enter data: 89
Enter data: 78
Linked list created successfully.
```

EXPLAINATION:

This Python program implements a Singly Linked List (SLL) using classes and nodes.

Node stores the data and a link to the next node.

create() creates the linked list.

insert_beginning() and insert_end() add nodes at the beginning and end.

insert_at_index() inserts a node at a specific position.

delete_by_value() deletes a node with a given value.

delete_first() and delete_last() remove the first and last nodes.

count_nodes() counts the number of nodes.

display() traverses and displays the list.

A menu-driven loop allows the user to choose operations until Exit.

B.AIM:

Write a program to implement DLL with the following operations.

1. create linked list
2. insert beginning
3. insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/traverse
10. Exit

PROGRAM:

class Node: def **init**(self, data): self.data = data self.prev = None self.next = None

class DoublyLinkedList: def **init**(self): self.head = None

```
# 1. Create linked list
def create(self):
    n = int(input("Enter number of nodes: "))
for i in range(n):

data = int(input("Enter data: "))
       new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp
```

```
    print("Doubly linked list created successfully.")

# 2. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node

    print("Node inserted at beginning.")

# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
 while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    print("Node inserted at end.")

# 4. Insert at index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))

    if index < 0:
```

```
            print("Invalid index.")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head

            if self.head:
                self.head.prev = new_node

            self.head = new_node
            print("Node inserted.")
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range.")
            return

        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted.")

    # 5. Delete by value
```

```
def delete_by_value(self):
    value = int(input("Enter value to delete: "))

    temp = self.head

    while temp and temp.data != value:
        temp = temp.next

    if temp is None:
        print("Value not found.")
        return

    if temp.prev:
        temp.prev.next = temp.next
    else:
        self.head = temp.next

    if temp.next:
        temp.next.prev = temp.prev

    print("Node deleted.")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next

    if self.head:
        self.head.prev = None

    print("First node deleted.")

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
```

```
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            print("Last node deleted.")
            return

        while temp.next:
            temp = temp.next

        temp.prev.next = None

        print("Last node deleted.")

    # 8. Count number of nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Doubly Linked List:", end=" ")

        while temp:
            print(temp.data, end=" <-> ")
```

```
        temp = temp.next

    print("None")
```

dll = DoublyLinkedList()

while True: print("\n----- DOUBLY LINKED LIST -----") print("1. Create linked list") print("2. Insert at beginning") print("3. Insert at end") print("4. Insert at index") print("5. Delete by value") print("6. Delete first node") print("7. Delete last node") print("8. Count number of nodes") print("9. Display / Traverse") print("10. Exit")

```
choice = int(input("Enter your choice: "))
if choice == 1:
    dll.create()

elif choice == 2:
    dll.insert_beginning()

elif choice == 3:
    dll.insert_end()

elif choice == 4:
    dll.insert_at_index()

elif choice == 5:
    dll.delete_by_value()

elif choice == 6:
    dll.delete_first()

elif choice == 7:
    dll.delete_last()

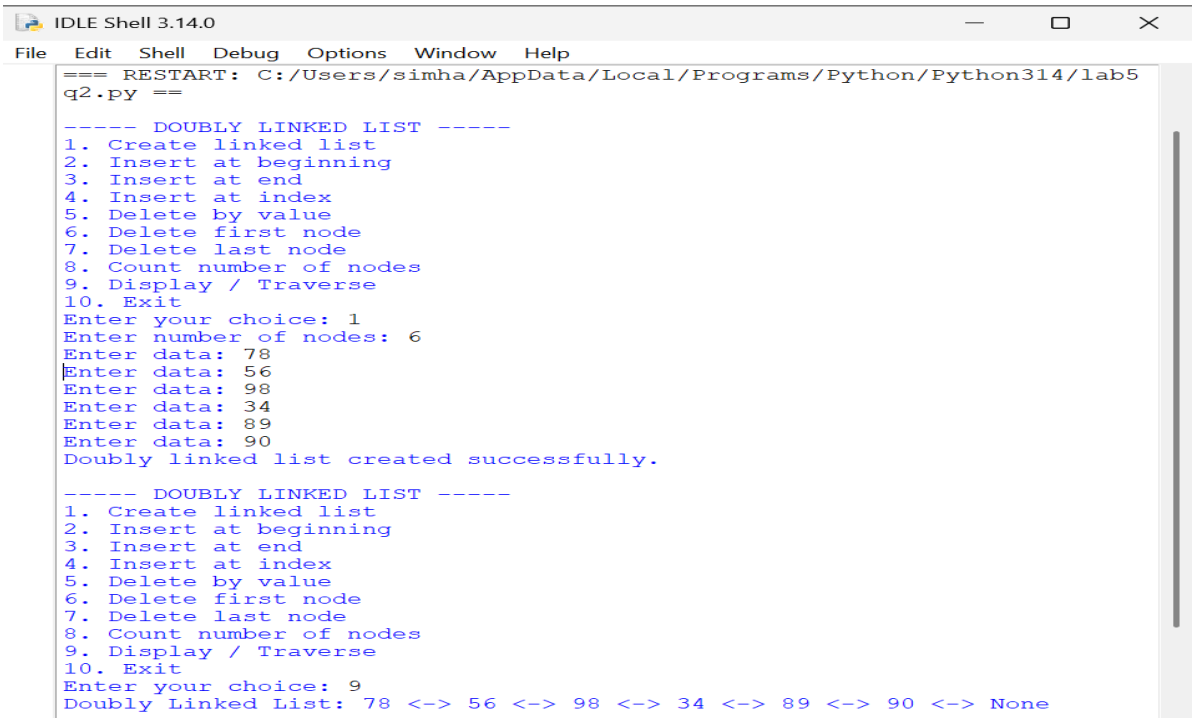
elif choice == 8:
    dll.count_nodes()

elif choice == 9:
    dll.display()
```

```
elif choice == 10:
    print("Program terminated.")
    break

else:
    print("Invalid choice. Please try again.
```

OUTPUT:





EXPLAINATION:

This Python program implements a **Doubly Linked List (DLL)** using classes and nodes.

- Each node contains **data, `prev`, and `next`** links.
- `create()` creates the linked list.
- `insert_beginning()`, `insert_end()`, and `insert_at_index()` insert nodes.
- `delete_by_value()`, `delete_first()`, and `delete_last()` delete nodes.
- `count_nodes()` counts the nodes.
- `display()` traverses and displays the list.
- A **menu-driven loop** allows the user to select operations until Exit

C.AIM:

Write a program to implement CLL with the following operations.

1. create linked list
2. insert beginning
3. insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/traverse
10. Exit

PROGRAM:

class Node: def **init**(self, data): self.data = data self.next = None

class CircularLinkedList: def **init**(self): self.head = None

```
# 1. Create linked list
def create(self):
    n = int(input("Enter number of nodes: "))

    for i in range(n):
        data = int(input("Enter data: "))
       new_node = Node(data)

       if self.head is None:
            self.head = new_node
            new_node.next = self.head
```

```
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head

    print("Circular linked list created successfully.")

# 2. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    print("Node inserted at beginning.")

# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
```

```
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head

    print("Node inserted at end.")

# 4. Insert at index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))

    if index < 0:
        print("Invalid index.")
        return

    if index == 0:
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

        print("Node inserted.")
        return
```

```
    if self.head is None:
        print("Index out of range.")
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Index out of range.")
            return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

    print("Node inserted.")

# 5. Delete by value
def delete_by_value(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("List is empty.")
        return

    current = self.head
    previous = None

    while True:
        if current.data == value:
            # Only one node
            if current == self.head and current.next == self.head:
                self.head = None

            # Delete head
            elif current == self.head:
                last = self.head
```

```
                while last.next != self.head:
                    last = last.next

                self.head = self.head.next
                last.next = self.head

            # Delete other node
            else:
                previous.next = current.next

            print("Node deleted.")
            return

        previous = current
        current = current.next

        if current == self.head:
            break

    print("Value not found.")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    # Only one node
    if self.head.next == self.head:
        self.head = None
    else:
        last = self.head

        while last.next != self.head:
            last = last.next

        self.head = self.head.next
        last.next = self.head
```

```
    print("First node deleted.")

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    # Only one node
    if self.head.next == self.head:
        self.head = None
    else:
        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head

    print("Last node deleted.")

# 8. Count number of nodes
def count_nodes(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 1
    temp = self.head.next

    while temp != self.head:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# 9. Display / Traverse
def display(self):
```

```
    if self.head is None:
        print("List is empty.")
        return

    temp = self.head

    print("Circular Linked List:", end=" ")

    while True:
        print(temp.data, end=" -> ")
        temp = temp.next

        if temp == self.head:
            break

    print("(back to head)")
```

cll = CircularLinkedList()

while True: print("\n----- CIRCULAR LINKED LIST -----") print("1. Create linked list") print("2. Insert at beginning") print("3. Insert at end") print("4. Insert at index") print("5. Delete by value") print("6. Delete first node") print("7. Delete last node") print("8. Count number of nodes") print("9. Display / Traverse") print("10. Exit")

```
choice = int(input("Enter your choice: "))
if choice == 1:
    cll.create()

elif choice == 2:
    cll.insert_beginning()

elif choice == 3:
    cll.insert_end()

elif choice == 4:
    cll.insert_at_index()

elif choice == 5:
    cll.delete_by_value()
```

```
elif choice == 6:
    cll.delete_first()

elif choice == 7:
    cll.delete_last()

elif choice == 8:
    cll.count_nodes()

elif choice == 9:
    cll.display()

elif choice == 10:
    print("Program terminated.")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab5q3.py ==

----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 34
Enter data: 65
Enter data: 98
Circular linked list created successfully.

----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated.
>>>
```

EXPLAINATION:

This Python program implements a **Circular Linked List (CLL)** using classes and nodes.

- Each node contains **data** and a `next` link.
- The last node always points back to the **head**.
- `create()` creates the linked list.
- `insert_beginning()`, `insert_end()`, and `insert_at_index()` add nodes.
- `delete_by_value()`, `delete_first()`, and `delete_last()` remove nodes.
- `count_nodes()` counts the nodes.
- `display()` traverses the list until it reaches the head again.
- The **menu-driven loop** allows the user to perform operations until Exit

WEEK 6

A.AIM:

Write a Python program to implement stack operations such as push,pop,display using an array.

PROGRAM:

```
stack = []
max_size = int(input("Enter stack size: "))

def push():
    if len(stack) == max_size:
        print("Stack Overflow")
    else:
        element = int(input("Enter element: "))
        stack.append(element)
        print("Element pushed successfully")

def pop():
    if len(stack) == 0:
        print("Stack Underflow")
    else:
        print("Popped element:", stack.pop())

def display():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Stack elements:", stack[::-1])
```

```
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        display()
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab6q1.py ==
Enter stack size: 5

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 45
Element pushed successfully

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 45

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 6
Invalid choice

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
```

EXPLAINATION:

- Push: Adds an element to the top of the stack.
- Pop: Removes the top element.
- Display: Shows all stack elements from top to bottom.
- Stack follows LIFO (Last In, First Out).
- Overflow occurs when the stack is full.
- Underflow occurs when trying to pop from an empty stack.

ERRORTABLE:

Sno:	Error	Rectification
1.	ValueError	Enter a valid integer
2.	IndexError	Check whether the stack is empty first

B.AIM:

Write a Python program to implement stack operations such as push,pop,display using a linked list.

PROGRAM:

class Node: def **init**(self, data): self.data = data self.next = None

class Stack: def **init**(self): self.top = None

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    print("Element pushed successfully")

def pop(self):
    if self.top is None:
        print("Stack Underflow")
    else:
        print("Popped element:", self.top.data)
        self.top = self.top.next

def display(self):
    if self.top is None:
        print("Stack is empty")
```

```
    else:
        temp = self.top
        print("Stack elements:")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next
        print()
```

stack = Stack()

while True: print("stack operations using lincked list") print("\n1. Push") print("2. Pop") print("3. Display") print("4. Exit")

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element: "))
    stack.push(data)
elif choice == 2:
    stack.pop()
elif choice == 3:
    stack.display()
elif choice == 4:
    break
else:
    print("Invalid choice")
```

Output:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab6q2.py ==
stack operations using lincked list

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 54
Element pushed successfully
stack operations using lincked list

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 54
stack operations using lincked list

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
```

EXPLAINATION:

- Node: Stores data and the address of the next node.
- Push: Adds a new node at the top.
- Pop: Removes the top node.
- Display: Traverses and prints all nodes.
- Stack follows LIFO (Last In, First Out).
- Underflow occurs when pop is performed on an empty stack

ERRORTABLE:

Sno	**Error**	**Rectification**
1.	`ValueError`	Enter a valid integer
2.	`AttributeError`	Check whether the stack is empty before accessing the node

WEEK 7

A.AIM:

Write a Python program to implement queue and its operations such as enqueue, dequeue, heap and display with a menu driven approach, implement using array data structures.

PROGRAM:

```
queue = []
max_size = int(input("Enter queue size: "))


while True:
    print("menu")
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(queue) == max_size:
            print("Queue Overflow")
        else:
            data = int(input("Enter element: "))
            queue.append(data)
            print("Element inserted")

    elif choice == 2:
```

```
        if len(queue) == 0:
            print("Queue Underflow")
        else:
            print("Deleted element:", queue.pop(0))

    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:", queue[0])

    elif choice == 4:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Queue:", queue)

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
=== RESTART: C:/Users/simha/AppData/Local/Programs/Python/Python314/lab7q1.py ==
Enter queue size: 4
menu

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 65
Element inserted
menu

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 65
menu

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 65
Invalid choice
menu

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting...
```

EXPLAINATION:

- Enqueue: Adds an element at the rear of the queue.
- Dequeue: Removes the element from the front.
- Peek: Displays the front element without removing it.
- Display: Shows all elements in the queue.
- Queue follows FIFO (First In, First Out).
- The queue is implemented using a Python list (array).
- Overflow: Occurs when the queue is full.
- Underflow: Occurs when the queue is empty.
-

B. AIM:

Write a Python program to implement queue and its operations such as enqueue, dequeue, heap and display with a menu driven approach, implement using linked list data structures.

PROGRAM:

class Node: def **init**(self, data): self.data = data self.next = None

class Queue: def **init**(self): self.front = None self.rear = None

```
def enqueue(self, data):
    new_node = Node(data)

    if self.rear is None:
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node

    print("Element inserted")

def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        print("Deleted element:", self.front.data)
        self.front = self.front.next

        if self.front is None:
            self.rear = None

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        temp = self.front
```

```
        print("Queue:", end=" ")

        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()
```

q = Queue()

while True: print("\n1. Enqueue") print("2. Dequeue") print("3. Peek") print("4. Display") print("5. Exit")

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element: "))
    q.enqueue(data)

elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()

elif choice == 5:
    print("Exiting...")
    break

else:
    print("Invalid choice")
```

OUTPUT:



EXPLAINATION:

- **Node:** Stores data and the link to the next node.
- **Enqueue:** Adds a new node at the rear.
- **Dequeue:** Removes a node from the front.
- **Peek:** Displays the front element without removing it.
- **Display:** Traverses and displays all elements.
- Queue follows **FIFO (First In, First Out)**.
- `front` points to the first node and `rear` points to the last node

ERRORTABLE:

Sno:	**Error**	**Rectification**
1.	`ValueError`	Enter a valid integer
2.	`AttributeError`	Check whether the node exists before accessing it

WEEK 8

A.AIM: Write a Python program to implement a binary tree with its operations using an arrays

PROGRAM:

class BinaryTree: def **init**(self, size): self.tree = [None] * size self.size = size

```
def insert(self, value, index=0):
    if index >= self.size:
        print("Tree is full")
        return

    if self.tree[index] is None:
        self.tree[index] = value
        print("Node inserted")
    else:
        print("Position already occupied")

def delete(self, value):
    if value in self.tree:
        index = self.tree.index(value)
        self.tree[index] = None
        print("Node deleted")
    else:
        print("Node not found")

def display(self):
    print("Binary Tree:", self.tree)

def inorder(self, i=0):
    if i < self.size and self.tree[i] is not None:
        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

def preorder(self, i=0):
    if i < self.size and self.tree[i] is not None:
        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)
```

```
def postorder(self, i=0):
    if i < self.size and self.tree[i] is not None:
        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")
```

bt = BinaryTree(10)

while True: print("\n1.Insert") print("2.Delete") print("3.Display") print("4.Inorder") print("5.Preorder") print("6.Postorder") print("7.Exit")

```
ch = int(input("Enter choice: "))

if ch == 1:
    value = int(input("Enter value: "))
    bt.insert(value)

elif ch == 2:
    value = int(input("Enter value to delete: "))
    bt.delete(value)

elif ch == 3:
    bt.display()

elif ch == 4:
    bt.inorder()
    print()

elif ch == 5:
    bt.preorder()
    print()

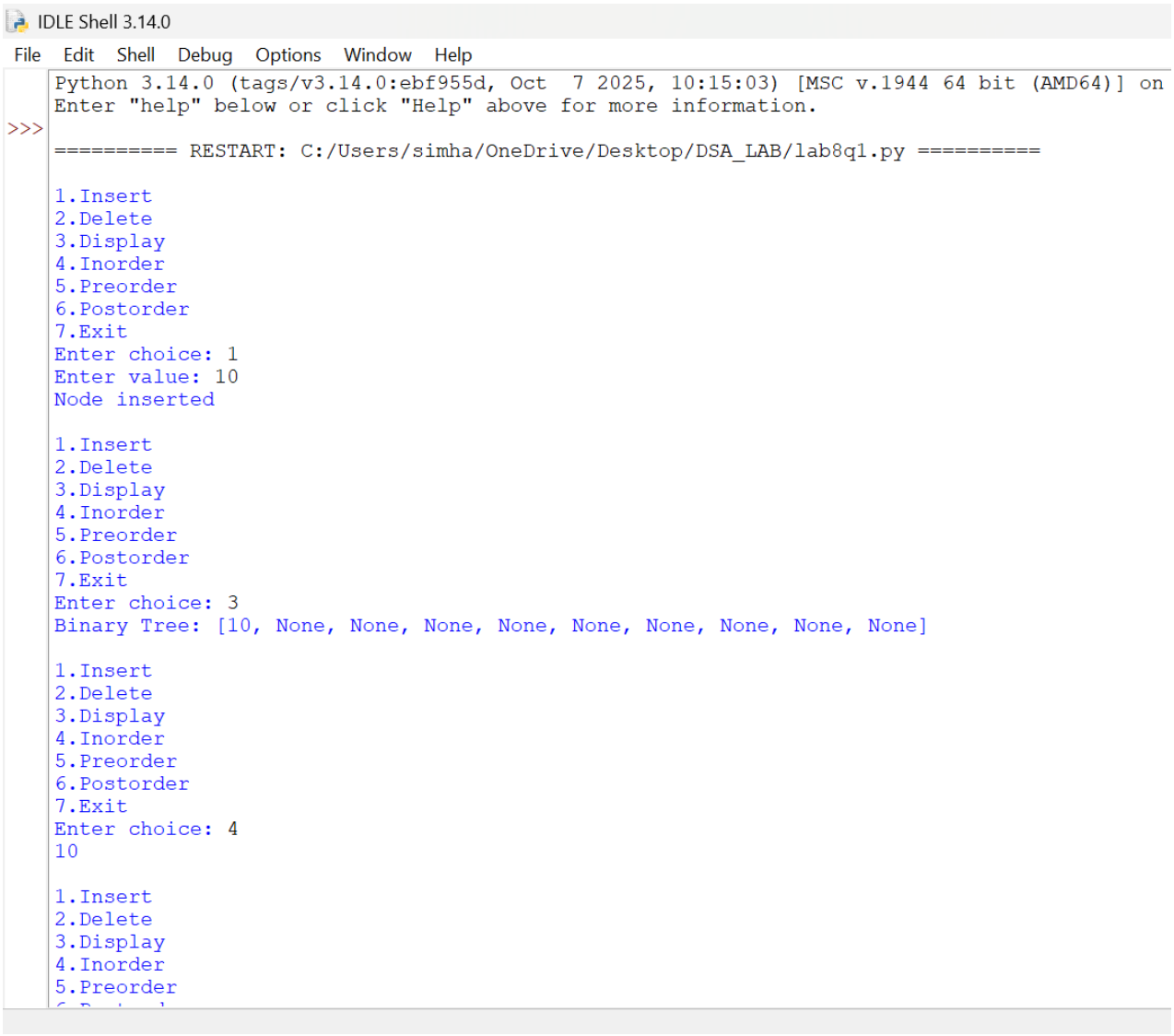
elif ch == 6:
    bt.postorder()
    print()

elif ch == 7:
    print("Exiting...")
```

```
    break

else:
    print("Invalid choice")
```

OUTPUT:





EXPLAINATION:

- Insert: Adds a node.
- Delete: Removes a node.
- Display: Shows tree elements.
- Inorder: Left → Root → Right.
- Preorder: Root → Left → Right.
- Postorder: Left → Right → Root.
- Array: Stores tree elements.
- Binary Tree: Each node has at most two children.

ERROR TABLE:

SNO:	ERROR	Rectification
1.	Valu error	Enter a valid integer
2.	Index error	Check array index limits

B. AIM: Write a Python program to implement a binary tree with its operations using an Linked list.

PROGRAM:

class Node: def **init**(self, data): self.data = data self.left = None self.right = None

class BinaryTree: def **init**(self): self.root = None

```
def insert(self, data):
    new_node = Node(data)

    if self.root is None:
        self.root = new_node
        return

    queue = [self.root]

    while queue:
        temp = queue.pop(0)

        if temp.left is None:
            temp.left = new_node
            return
        else:
            queue.append(temp.left)

        if temp.right is None:
            temp.right = new_node
            return
        else:
            queue.append(temp.right)

def inorder(self, node):
    if node:
```

```
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

def preorder(self, node):
    if node:
        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

def postorder(self, node):
    if node:
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

def display(self):
    if self.root is None:
        print("Tree is empty")
    else:
        print("Inorder: ", end="")
        self.inorder(self.root)

        print("\nPreorder: ", end="")
        self.preorder(self.root)

        print("\nPostorder: ", end="")
        self.postorder(self.root)
        print()
```

tree = BinaryTree()

while True: print("\n1. Insert") print("2. Display") print("3. Exit")

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element: "))
    tree.insert(data)
```

```
elif choice == 2:
    tree.display()

elif choice == 3:
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
Enter "help" below or click "Help" above for more info
>>>
========== RESTART: C:/Users/simha/OneDrive/Desktop/DS

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter element: 10

1. Insert
2. Display
3. Exit
Enter your choice: 2
Inorder: 10
Preorder: 10
Postorder: 10

1. Insert
2. Display
3. Exit
Enter your choice: 3
>>>
```

EXPLAINATION:

- Node: Stores data and links to left and right children.
- Insert: Adds a node at the first available position.
- Inorder: Left → Root → Right.
- Preorder: Root → Left → Right.
- Postorder: Left → Right → Root.
- The tree is implemented using linked nodes.

ERROR TABLE:

SNO:	**ERROR**	**Rectification**
1.	Valu error	Enter a valid integer
2.	Index error	Check array index limits

WEEK 9

A. AIM: Binary Search Trees Python program to implement Binary Search Tree (BST) operations.

PROGRAM:

class Node: def **init**(self, data): self.data = data self.left = None self.right = None

class BST: def **init**(self): self.root = None

```
def insert(self, root, data):
    if root is None:
        return Node(data)

   if data < root.data:
        root.left = self.insert(root.left, data)
    elif data > root.data:
        root.right = self.insert(root.right, data)

    return root

def search(self, root, key):
    if root is None or root.data == key:
        return root

    if key < root.data:
        return self.search(root.left, key)
    else:
        return self.search(root.right, key)

def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

def preorder(self, root):
    if root:
        print(root.data, end=" ")
```

```
        self.preorder(root.left)
        self.preorder(root.right)

def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

def find_min(self, root):
    while root.left:
        root = root.left
    return root

def delete(self, root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = self.delete(root.left, key)
    elif key > root.data:
        root.right = self.delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = self.find_min(root.right)
        root.data = temp.data
        root.right = self.delete(root.right, temp.data)

    return root


bst = BST()

while True: print("\n1. Insert") print("2. Search") print("3. Delete") print("4. Inorder") print("5. Preorder") print("6. Postorder") print("7. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element: "))
    bst.root = bst.insert(bst.root, data)

elif choice == 2:
    key = int(input("Enter element to search: "))
    if bst.search(bst.root, key):
        print("Element found")
    else:
        print("Element not found")

elif choice == 3:
    key = int(input("Enter element to delete: "))
    bst.root = bst.delete(bst.root, key)
    print("Deletion completed")

elif choice == 4:
    bst.inorder(bst.root)
    print()

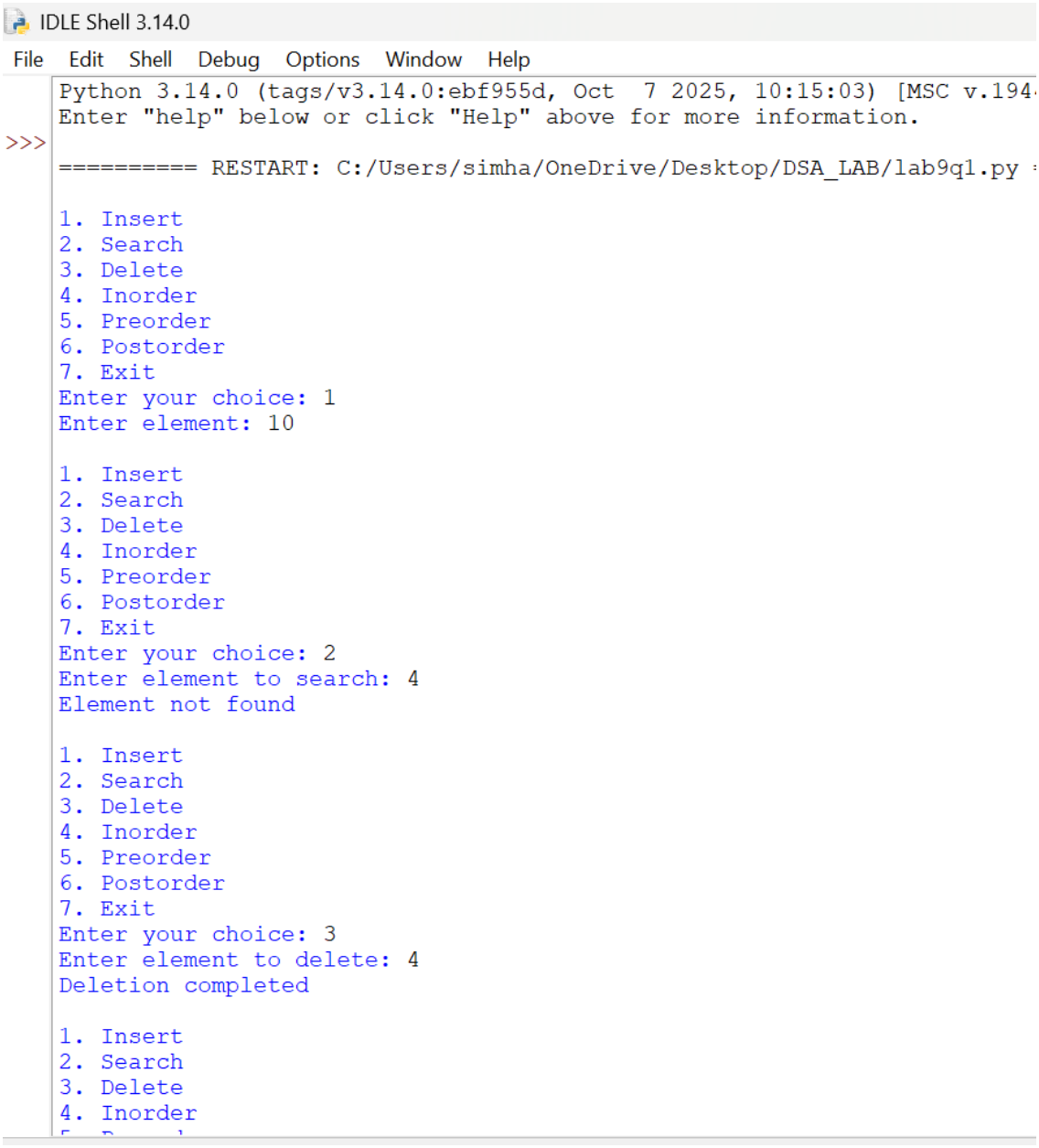
elif choice == 5:
    bst.preorder(bst.root)
    print()

elif choice == 6:
    bst.postorder(bst.root)
    print()

elif choice == 7:
    break

else:
    print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.194
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/simha/OneDrive/Desktop/DSA_LAB/lab9q1.py
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 1
Enter element: 10
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 2
Enter element to search: 4
Element not found
1. Insert
2. Search
3. Delete
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 3
Enter element to delete: 4
Deletion completed
1. Insert
2. Search
3. Delete
4. Inorder



EXPLAINATION:

- Node: Stores data and left/right links.
- Insert: Adds elements in BST order.
- Search: Finds a given element.
- Delete: Removes an element from the tree.
- Inorder, Preorder, Postorder: Display elements in different traversal orders.
- BST follows: Left child < Root < Right child.

ERROR TABLE:

SNO:	**ERROR**	**Rectification**
1.	Valu error	Enter a valid integer
2.	Index error	Use a balanced BST

WEEK 10

A. AIM: AVL Tree Write a Python program to implement an AVL Tree with all its operations.

PROGRAM:

class Node: def **init**(self, data): self.data = data self.left = None self.right = None self.height = 1

class AVLTree: def height(self, root): if root is None: return 0 return root.height

```
def balance(self, root):
    if root is None:
        return 0
    return self.height(root.left) - self.height(root.right)

def right_rotate(self, y):
    x = y.left
    t2 = x.right

   x.right = y
    y.left = t2

    y.height = 1 + max(self.height(y.left), self.height(y.right))
    x.height = 1 + max(self.height(x.left), self.height(x.right))

    return x

def left_rotate(self, x):
    y = x.right
    t2 = y.left

    y.left = x
    x.right = t2

    x.height = 1 + max(self.height(x.left), self.height(x.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))

    return y
```

```
def insert(self, root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = self.insert(root.left, data)
    elif data > root.data:
        root.right = self.insert(root.right, data)
    else:
        return root

    root.height = 1 + max(self.height(root.left),
self.height(root.right))
    b = self.balance(root)

    if b > 1 and data < root.left.data:
        return self.right_rotate(root)

    if b < -1 and data > root.right.data:
        return self.left_rotate(root)

    if b > 1 and data > root.left.data:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)

    if b < -1 and data < root.right.data:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)

    return root

def min_value(self, root):
    while root.left:
        root = root.left
    return root

def delete(self, root, data):
    if root is None:
        return root
```

```
    if data < root.data:
        root.left = self.delete(root.left, data)
    elif data > root.data:
        root.right = self.delete(root.right, data)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = self.min_value(root.right)
        root.data = temp.data
        root.right = self.delete(root.right, temp.data)

    if root is None:
        return root

    root.height = 1 + max(self.height(root.left),
self.height(root.right))
    b = self.balance(root)

    if b > 1 and self.balance(root.left) >= 0:
        return self.right_rotate(root)

    if b > 1 and self.balance(root.left) < 0:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)

    if b < -1 and self.balance(root.right) <= 0:
        return self.left_rotate(root)

    if b < -1 and self.balance(root.right) > 0:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)

    return root

def search(self, root, key):
```

```
    if root is None or root.data == key:
        return root

    if key < root.data:
        return self.search(root.left, key)
    return self.search(root.right, key)

def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")
```

avl = AVLTree() root = None

while True: print("\n1. Insert") print("2. Delete") print("3. Search") print("4. Inorder") print("5. Preorder") print("6. Postorder") print("7. Exit")

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element: "))
    root = avl.insert(root, data)

elif choice == 2:
    data = int(input("Enter element to delete: "))
    root = avl.delete(root, data)
```

```
elif choice == 3:
    data = int(input("Enter element to search: "))
    if avl.search(root, data):
        print("Element found")
    else:
        print("Element not found")

elif choice == 4:
    avl.inorder(root)
    print()

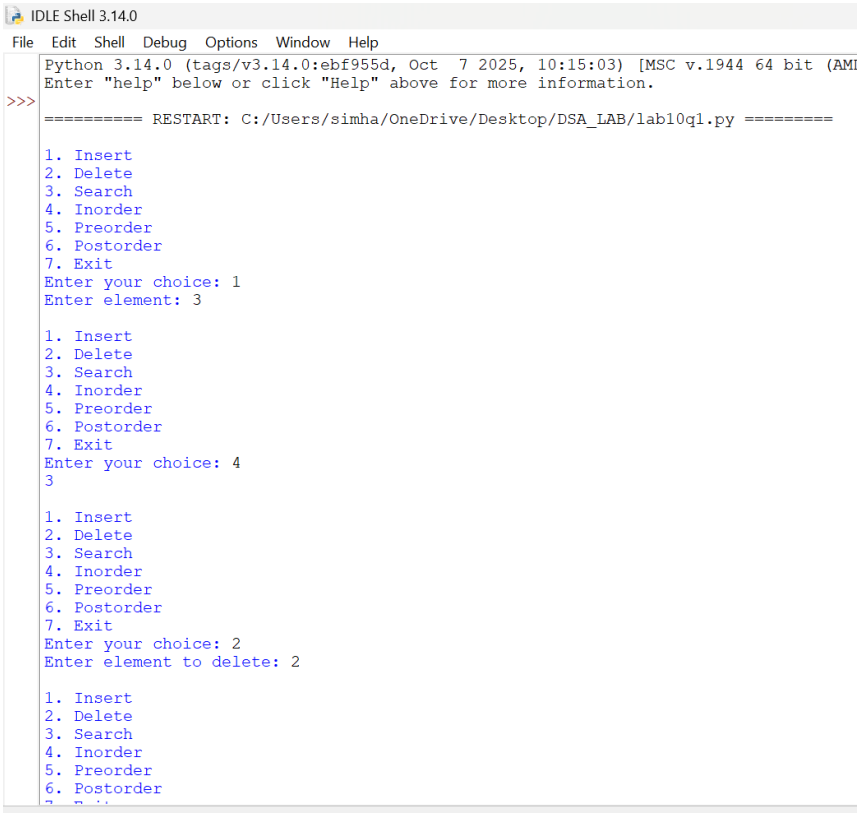
elif choice == 5:
    avl.preorder(root)
    print()

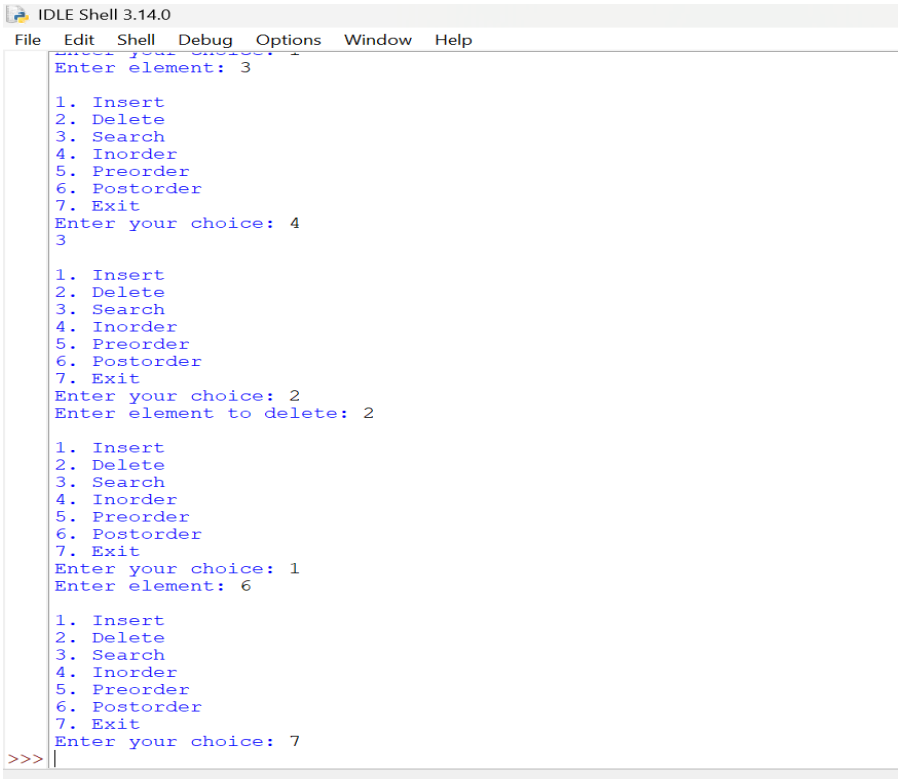
elif choice == 6:
    avl.postorder(root)
    print()

elif choice == 7:
    break

else:
    print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMI
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/simha/OneDrive/Desktop/DSA_LAB/lab10q1.py =========
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 1
Enter element: 3
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 4
3
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 2
Enter element to delete: 2
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter element: 3
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 4
3
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 2
Enter element to delete: 2
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 1
Enter element: 6
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter your choice: 7
>>>

EXPLAINATION:

- AVL Tree: A self-balancing Binary Search Tree.
- Insert/Delete: Adds or removes nodes while maintaining balance.
- Rotations: LL, RR, LR and RL rotations balance the tree.
- Search: Finds the required element.
- Traversals: Inorder, Preorder and Postorder display elements.
- Balance factor must be -1, 0 or +1.

WEEK 11

A. AIM: Hashing Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

PROGRAM:

```
size = 10

# Separate Chaining

def separate_chaining(keys):

    table = [[] for i in range(size)]

    for key in keys:

        index = key % size

        table[index].append(key)

    print("Separate Chaining:")

    for i in range(size):

        print(i, ":", table[i])

# Linear Probing

def linear_probing(keys):

    table = [None] * size

    for key in keys:

        index = key % size


        while table[index] is not None:

            index = (index + 1) % size

        table[index] = key

    print("Linear Probing:")

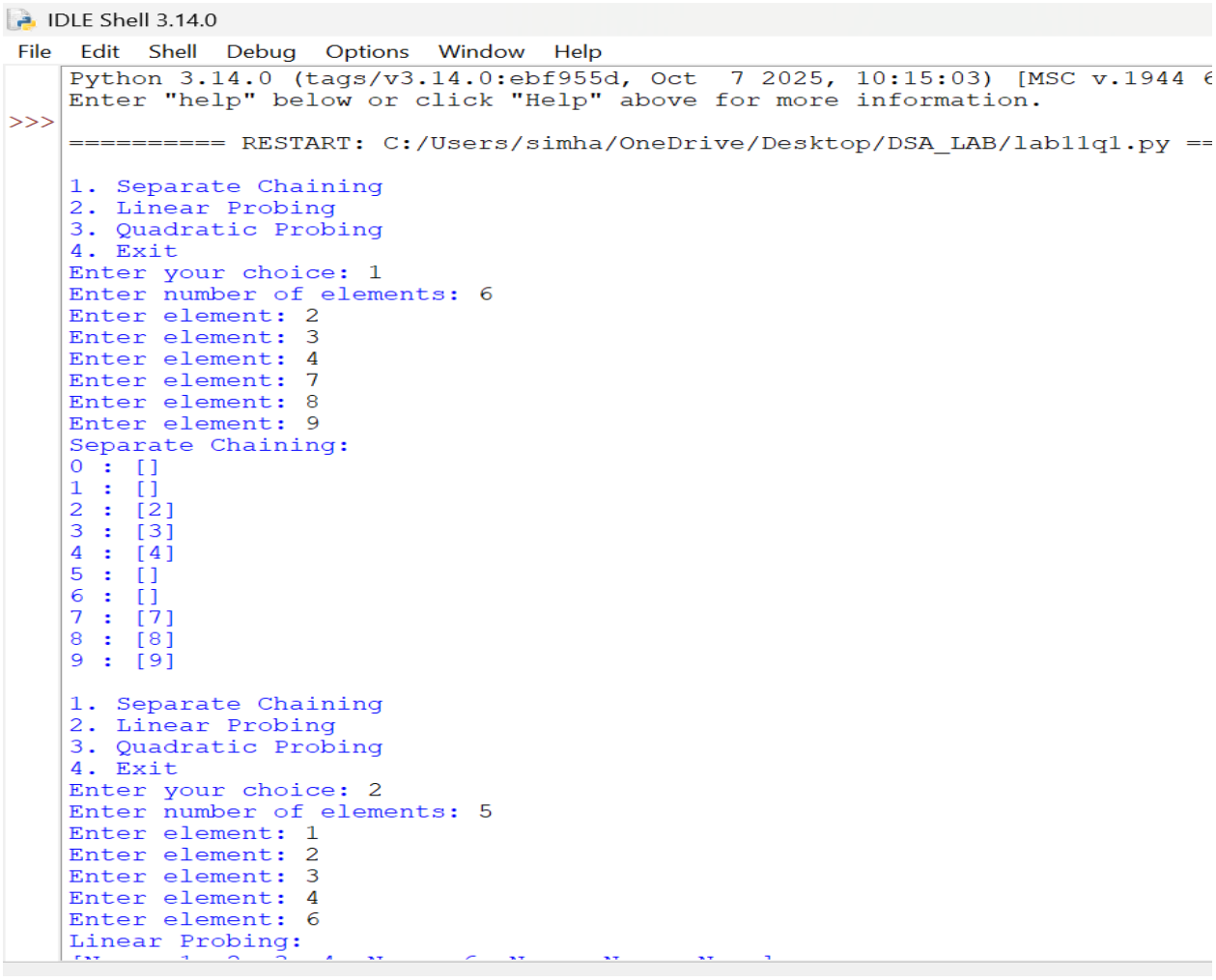
    print(table)

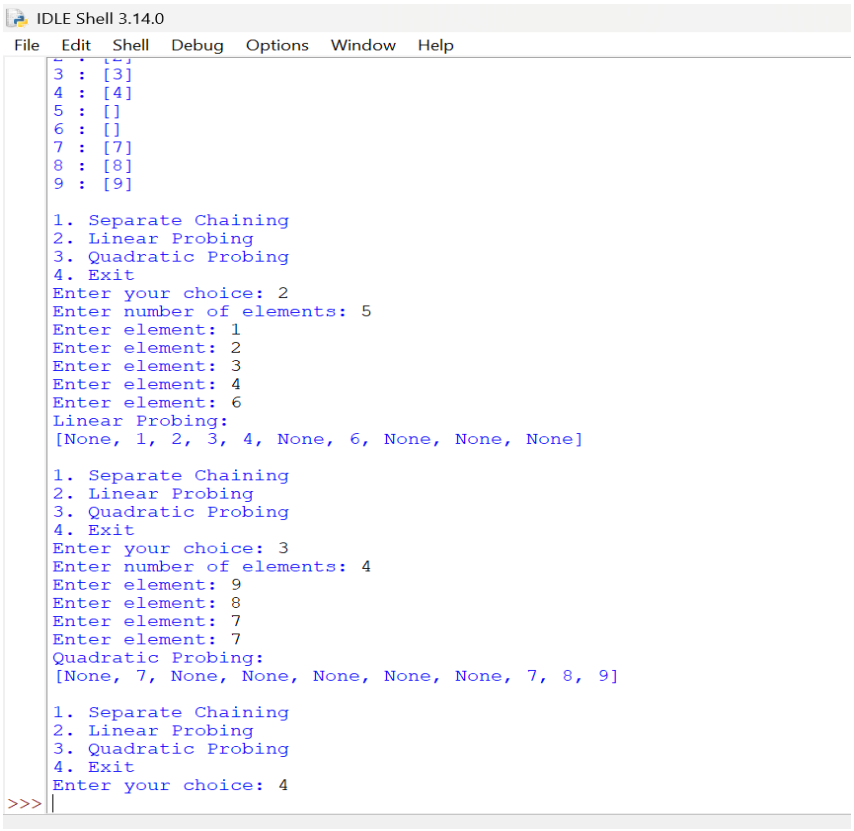
# Quadratic Probing
```

```
def quadratic_probing(keys):
    table = [None] * size
    for key in keys:
        index = key % size
        i = 0
        while i < size and table[(index + i*i) % size] is not None:
            i += 1
        if i < size:
            table[(index + i*i) % size] = key
        else:
            print("Hash table is full for", key)
    print("Quadratic Probing:")
    print(table)
while True:
    print("\n1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")
    choice = int(input("Enter your choice: ")
    if choice == 4:
        break
    n = int(input("Enter number of elements: "))
    keys = []
    for i in range(n):
        keys.append(int(input("Enter element: ")))
    if choice == 1:
        separate_chaining(keys)
```

```
    elif choice == 2:
        linear_probing(keys)
    elif choice == 3:
        quadratic_probing(keys)
    else:
        print("Invalid choice")
```

OUTPUT:





EXPLAINATION:

- Hashing: Stores data using a hash function (key % size).
- Separate Chaining: Stores collided elements in a list.
- Linear Probing: Checks the next available position.
- Quadratic Probing: Checks positions using squared increments.
- Collision: Occurs when two keys get the same index.

ERROR TABLE:

SNO:	**ERROR**	**Rectification**
1.	Valu error	Non-integer input is entered
2.	Index error	Accessing an invalid hash table index